Events - Handling, Bubbling & Event Delegation in JavaScript - Assignment Solutions

PW EARNERS : Coding and Web Development

By : Nishant Saini Sir

LEC - 40

Section 1 - Event Handling Basics

1. Handle a Button Click

Question:
Create a button and use addEventListener() to display a message when the button is clicked.
HTML:
<button id="myButton">Click Me</button>

Solution:
```
let button = document.getElementById("myButton");
button.addEventListener("click", function() {
    console.log("Button Clicked!");
});
```

Output:
Button Clicked!

---

2. Change Text on Click

Question:
Create a paragraph and a button. Use addEventListener() to change the paragraph text when the button is clicked.
HTML:
Welcome to my website!
<button id="myButton">Click Me</button>

Solution:
```
let message = document.getElementById("message");
let button = document.getElementById("myButton");
button.addEventListener("click", function() {
    message.textContent = "Thanks for visiting!";
});
```

Output:
Before clicking:
Welcome to my website!
After clicking:
Thanks for visiting!

---

3. Handle a Mouseover Event

Question:
Create a heading and use addEventListener() with the mouseover event to change its text when the mouse moves over it.
HTML:
<h1 id="heading">JavaScript</h1>

Solution:
```
let heading = document.getElementById("heading");
heading.addEventListener("mouseover", function() {
    heading.textContent = "Mouse is over the heading!";
});
```

Output:
Before mouseover:
JavaScript
After mouseover:
Mouse is over the heading!

Section 2 - Event Object

4. Display the Clicked Element

Question:
Create a button and use the event object to identify the element that was clicked.
HTML:
<button id="myButton">Click Me</button>

Solution:
```
let button = document.getElementById("myButton");
button.addEventListener("click", function(event) {
    console.log(event.target.tagName);
});
```

Output:
BUTTON

---

5. Display Mouse Coordinates

Question:

Create a and use a mousemove event to display the mouse coordinates using the event object's clientX and clientY properties.

HTML:

```
<div id="box" style="width: 400px; height: 300px; border: 1px solid black;">
   Move your mouse here
</div>
<p id="coordinates"></p>
```

Solution:

```
let box = document.getElementById("box");
let coordinates = document.getElementById("coordinates");
box.addEventListener("mousemove", function(event) {
   coordinates.textContent = `X: ${event.clientX} Y: ${event.clientY}`;
});
```

Output:

X: 250 Y: 180

The exact X and Y values will depend on the current mouse position.

---

6. Get the Value of an Input Using the Event Object

Question:

Create an input field and use the event object's target.value to display the entered value.

HTML:

```
<input type="text" id="inputBox" placeholder="Type something">
<p id="result"></p>
```

Solution:

```
let inputBox = document.getElementById("inputBox");
let result = document.getElementById("result");
inputBox.addEventListener("input", function(event) {
   result.textContent = `You typed: ${event.target.value}`;
});
```

Output:

If the user enters:

JavaScript

The output will be:

You typed: JavaScript

Section 3 - Removing and Controlling Events

7. Remove an Event Listener

Question:
Create a button and attach a click event using addEventListener(). Create a separate function and use removeEventListener() to stop the click event when required.
HTML:
<button id="myButton">Click Me</button>
<button id="removeButton">Remove Event</button>

Solution:
```
let button = document.getElementById("myButton");
let removeButton = document.getElementById("removeButton");
function showMessage() {
    console.log("Button clicked!");
}
button.addEventListener("click", showMessage);
removeButton.addEventListener("click", function() {
    button.removeEventListener("click", showMessage);
});
```

Output:
Before removing the listener:
Button clicked!
After clicking Remove Event, clicking the first button will no longer display the message.

---

8. Run an Event Only Once

Question:
Create a button and use addEventListener() with the once option so that the event runs only the first time the button is clicked.
HTML:
<button id="myButton">Click Me</button>

Solution:
```
let button = document.getElementById("myButton");
button.addEventListener("click", function() {
    console.log("Welcome!");
}, { once: true });
```

Output:
First click:
Welcome!
Second click:
No message will be displayed.

---

9. Stop Event Propagation

Question:
Create a parent containing a button. Add click events to both parent and button. Use stopPropagation() so that clicking the button does not trigger the parent's click event.
HTML:

```
<div id="parent">
   <button id="myButton">Click Me</button>
</div>
```

Solution:

```
let parent = document.getElementById("parent");
let button = document.getElementById("myButton");
parent.addEventListener("click", function() {
   console.log("Parent clicked");
});
button.addEventListener("click", function(event) {
   event.stopPropagation();
   console.log("Button clicked");
});
```

Output with stopPropagation():
Button clicked
The "Parent clicked" message will not be displayed.

Section 4 - Bubbling, Capturing & Default Actions

10. Demonstrate Event Bubbling

Question:
Create a parent containing a child button. Add click events to both elements and observe the order in which the events execute when the button is clicked.
HTML:

```
<div id="parent">
   <button id="myButton">Click Me</button>
```

```
</div>
```

Solution:

```
let parent = document.getElementById("parent");
let button = document.getElementById("myButton");
button.addEventListener("click", function() {
   console.log("Button clicked");
});
parent.addEventListener("click", function() {
   console.log("Parent clicked");
});
```

Output:

Button clicked
Parent clicked
Explanation:
In event bubbling, the event starts from the target element and then moves upward to its parent.

---

11. Demonstrate Event Capturing

Question:

Create a parent containing a button. Add click event listeners to both using the capturing phase and observe the order in which the events execute.

HTML:

```
<div id="parent">
   <button id="myButton">Click Me</button>
</div>
```

Solution:

```
let parent = document.getElementById("parent");
let button = document.getElementById("myButton");
parent.addEventListener("click", function() {
   console.log("Parent clicked");
}, true);
button.addEventListener("click", function() {
   console.log("Button clicked");
}, true);
```

Output:
Parent clicked
Button clicked
Explanation:
In event capturing, the event travels from the parent toward the target element.

Section 5 - Event Delegation

12. Handle Multiple Buttons Using Event Delegation

Question:
Create multiple buttons inside a parent . Add only one event listener to the parent and use event delegation to identify which button was clicked.
HTML:

```
<div id="buttons">
   <button>HTML</button>
   <button>CSS</button>
   <button>JavaScript</button>
</div>
```

Solution:

```
let buttons = document.getElementById("buttons");
buttons.addEventListener("click", function(event) {
   if (event.target.tagName === "BUTTON") {
      console.log(`${event.target.textContent} button clicked`);
   }
});
```

Output when JavaScript button is clicked:
JavaScript button clicked

---

13. Handle a Dynamic List Using Event Delegation

Question:
Create a list of items and add one event listener to the <ul>. Use event delegation to display the text of the clicked list item.
HTML:

```
<ul id="list">
   <li>HTML</li>
   <li>CSS</li>
```

```
    <li>JavaScript</li>
</ul>
```

Solution:

```
let list = document.getElementById("list");
list.addEventListener("click", function(event) {
    if (event.target.tagName === "LI") {
        console.log(`You clicked: ${event.target.textContent}`);
    }
});
```

Output when CSS is clicked:

You clicked: CSS